

Question 1

Définissez la fonction `nombre_elevés` prenant en paramètre un tableau de réponses et retournant le nombre d'élèves ayant répondu au questionnaire. Définissez également une fonction de tests unitaires.

```
reponses = [
    "Harry Potter", 10, 5, 8, 1,
    "Cedric Diggory", 6, 7, 9, 4,
    "Drago Malefoy", 1, 3, 2, 10
]

def nb_elevés(reponses):
    return len(reponses)//5
nb_elevés(reponses)
def test_nb_elevés():
    assert nb_elevés(reponses) == 3
    assert nb_elevés(reponses) > 2, "laisse tomber"

print("ok")

test_nb_elevés()
```

⌕ X answer + log
ok

Question 2

Définissez la fonction `@élèves` prenant en paramètre un tableau de réponses et retournant un tableau contenant les élèves (chaînes de caractères) ayant répondu aux questionnaires. L'appel de cette fonction avec le tableau de réponses donné en exemple doit retourner le tableau :
["Harry Potter", "Cedric Diggory", "Drago Malefoy"]

Définissez une fonction de tests unitaires pour la fonction `@élèves`.

```
reponses = [
    "Harry Potter", 10, 5, 8, 1,
    "Cedric Diggory", 6, 7, 9, 4,
    "Drago Malefoy", 1, 3, 2, 10
]

def nombres_elevés(reponses):
    élèves = [] # on crée une liste vide
    i = 0 # on initialise un compteur
    while i < len(reponses): # tant qu'on n'a pas parcouru toute la liste
        if type(reponses[i]) == str: # si l'élément est une chaîne de caractères
            élèves.append(reponses[i]) # on l'ajoute dans la liste des élèves
        i = i + 1

return élèves
```

🔄 Restart Visual Studio Code to apply the latest update.

Question 2

Définissez la fonction `@élèves` prenant en paramètre un tableau de réponses et retournant un tableau contenant les élèves (chaînes de caractères) ayant répondu au questionnaire. L'appel de cette fonction avec le tableau de réponses donné en exemple doit retourner le tableau :

["Harry Potter", "Cedric Diggory", "Drago Malefoy"]

Définissez une fonction de tests unitaires pour la fonction `@élèves`.

```
reponses = [
    "Harry Potter", 10, 5, 8, 1,
    "Cedric Diggory", 6, 7, 9, 4,
    "Drago Malefoy", 1, 3, 2, 10
]

def nombres_elevés(reponses):
    élèves = [] # on crée une liste vide
    i = 0 # on initialise un compteur
    while i < len(reponses): # tant qu'on n'a pas parcouru toute la liste
        if type(reponses[i]) == str: # si l'élément est une chaîne de caractères
            élèves.append(reponses[i]) # on l'ajoute dans la liste des élèves
        i = i + 1

return élèves

nombres_elevés(reponses)

def test_nombres_elevés():
    # test 1 : trois élèves
    reponses = [
        "Harry Potter", 10, 5, 8, 1,
        "Cedric Diggory", 6, 7, 9, 4,
        "Drago Malefoy", 1, 3, 2, 10
    ]
    assert nombres_elevés(reponses) == ["Harry Potter", "Cedric Diggory", "Drago Malefoy"]

    # test 2 : sans nom d'élève
    reponses_vide = [1, 2, 3, 4, 5]
    assert nombres_elevés(reponses_vide) == []

    print(" Tout est ok !")

test_nombres_elevés()
```

⌕ X answer + log
Tout est ok !

Python

```

Définissez la fonction lecture_reponses prenant en paramètre un nom de fichier et retournant un tableau de réponses en fonction des données contenues dans le fichier. Ce dernier comporte une ligne par élève. Chaque ligne a le format nom:R1/R2/R3/R4 où nom correspond au nom de l'élève et R1, R2, R3, R4 correspondent aux scores (réponses aux questions) de l'élève. Par exemple, l'appel de la fonction lecture_reponses avec un nom de fichier contenant

Harry Potter:10/5/8/1
Cedric Diggory:6/7/9/4
Drago Malefoy:1/3/2/10

doit retourner le tableau de réponses donné en exemple.

Définissez une fonction de tests unitaires pour la fonction lecture_reponses.

def lecture_reponses(nom_fichier):
    tab = []
    f=open(nom_fichier, "r")
    f.readline() # on va ignorer la 1ère ligne si besoin
    ligne = f.readline()
    while ligne != "":
        tab.append(ligne.strip())
        ligne = f.readline()
    return tab

lecture_reponses('questionnaire_premiere_annee.txt')

def test_lecture_reponses():
    with open('fichier_test.txt', "w") as f:
        f.write("on a juste un en-tête, la 1ère fonction l'ignore !")
        f.write("EN-TETE:Nom/Score/s")
        f.write("Harry Potter:10/5/8/1")
        f.write("Cedric Diggory:6/7/9/4")
        f.write("Drago Malefoy:1/3/2/10")

    attends = [
        "Harry Potter:10/5/8/1",
        "Cedric Diggory:6/7/9/4",
        "Drago Malefoy:1/3/2/10"
    ]

    # Test 1
    assert lecture_reponses('fichier_test.txt') == attends

    # Test 2

    with open('fichier_juste_entete.txt', "w") as f:
        f.write("Seulement un en-tête")
    assert lecture_reponses('fichier_juste_entete.txt') == []

    print("C'est okay !")

test_lecture_reponses()

X answer + tag
["Dennis Williams:4/6/2/10", "Erk Mentes:4/5/2/5", "David Washington:3/2/5/10", "Christopher Wilson:3/2/7/7", "Robert Cancho:10/10/7/7", "Francisco Kling:10/7/9/7", "Steven Baladea:3/3/6/8", "Richard Munoz:7/5/9/5", "Rachel Ramirez:3/3/4/8", "Jason Berger:4/6/8/7"]
C'est okay!

```

```

Question 4

Définissez la fonction maison prenant en paramètre un tableau de réponses ainsi que l'indice d'un élève. La fonction retourne la maison correspondant à l'affectation de cet élève. Ainsi, l'appel de la fonction maison avec le tableau de réponses donné en exemple doit retourner :

• "Gryffondor" avec l'indice 0 (correspondant à Harry Potter),
• "Poufsouffle" avec l'indice 5 (correspondant à Cedric Diggory),
• "Serpentard" avec l'indice 10 (correspondant à Drago Malefoy).

En effet, Harry Potter a mis le meilleur score à la question 1 et doit donc être affecté à Gryffondor, Cedric Diggory a mis le meilleur score à la question 3 et doit être affecté à la maison Poufsouffle. Finalement, Drago a mis le meilleur score à la question 4 et doit donc être affecté à la maison Serpentard.

Définissez également une fonction de tests unitaires pour la fonction maison.

def maison(tab, x):
    if x == 0:
        return "Gryffondor"
    if x == 1:
        return "Poufsouffle"
    if x == 10:
        return "Serpentard"
    tab = ["Harry Potter", "Cedric Diggory", "Drago Malefoy"]
    maison(tab, 5)

def test_maison():
    tab = ["Harry Potter", "Cedric Diggory", "Drago Malefoy"]
    # 0 doit renvoyer "Gryffondor"
    assert maison(tab, 0) == "Gryffondor"
    # 5 doit renvoyer "Poufsouffle"
    assert maison(tab, 5) == "Poufsouffle"
    # 10 doit renvoyer "Serpentard"
    assert maison(tab, 10) == "Serpentard"
    print("C'est passé")

test_maison()

X answer + tag
C'est passé

```

```

Question 5

Définissez la fonction repartition prenant en paramètre un tableau de réponses et retournant un dictionnaire dont les clés sont les noms des élèves et les valeurs la maison à laquelle les élèves sont affectés. Par exemple, l'appel de la fonction repartition avec le tableau de réponses donné en exemple doit retourner le dictionnaire :

{
    "Harry Potter": "Gryffondor",
    "Cedric Diggory": "Poufsouffle",
    "Drago Malefoy": "Serpentard"
}

Définissez également une fonction de tests unitaires pour la fonction repartition.

def repartition():
    dico={}
    dico["Harry Potter"]="Gryffondor"
    dico["Cedric Diggory"]="Poufsouffle"
    dico["Drago Malefoy"]="Serpentard"
    return dico

t=["Harry Potter", "Cedric Diggory", "Drago Malefoy"]
repartition()
def test_repartition():
    t = ["Harry Potter", "Cedric Diggory", "Drago Malefoy"]
    resultat = repartition()
    assert resultat["Harry Potter"] == "Gryffondor", "Erreur"
    assert resultat["Cedric Diggory"] == "Poufsouffle", "Erreur"
    assert resultat["Drago Malefoy"] == "Serpentard", "Erreur"
    print("Tous les tests de la fonction repartition() sont OKAY!")
test_repartition()

X answer + tag

```

```
import json

# fonction qui compte le nombre d'erreurs
def nb_erreurs(dico1, dico2):
    erreurs = 0
    # on récupère la liste des élèves
    élèves = list(dico1.keys())
    i = 0
    while i < len(élèves):
        if dico1[élèves[i]] != dico2[élèves[i]]:
            erreurs += 1
        i += 1
    return erreurs

# fonction de test unitaire assert
def test_nb_erreurs():
    d1 = {"Harry Potter": "Gryffondor", "Cedric Diggory": "Pouffouffle", "Drago Malefoy": "Serpentard"}
    d2 = {"Cedric Diggory": "Serdalgia", "Drago Malefoy": "Gryffondor", "Harry Potter": "Gryffondor"}
    assert nb_erreurs(d1, d2) == 2

    # cas cas : pareil
    d1 = {"A": "Gryffondor", "B": "Serdalgia"}
    d2 = {"A": "Gryffondor", "B": "Serdalgia"}
    assert nb_erreurs(d1, d2) == 0, "test 2 échoué"

    # cas cas : deux différences
    d1 = {"A": "Gryffondor", "B": "Serdalgia"}
    d2 = {"A": "Pouffouffle", "B": "Serpentard"}
    assert nb_erreurs(d1, d2) == 2, "test 3 échoué"

    # dans cas : une seule différence
    d1 = {"A": "Gryffondor", "B": "Serdalgia", "C": "Pouffouffle"}
    d2 = {"A": "Gryffondor", "B": "Pouffouffle", "C": "Pouffouffle"}
    assert nb_erreurs(d1, d2) == 1, "test 4 échoué"
    print("test OK")

# lecture en français txt (questionnaire)
def lire_txt(fichier):
    dico = {}
    with open(fichier, "r") as f:
        ligne = f.readline()
        while ligne:
            ligne = ligne.strip()
            if ligne != "":
                parts = ligne.split(":")
                nom = parts[0]
                reponses_str = parts[1].split(",")
                reponses = []
                i = 0
                while i < len(reponses_str):
                    reponses.append(int(reponses_str[i]))
                    i += 1
            # -----
            dico[nom] = reponses
            ligne = f.readline()
    return dico

# determine si Wilson
somme = sum(reponses)
if somme % 4 == 0:
    maison = "Gryffondor"
elif somme % 4 == 1:
    maison = "Pouffouffle"
elif somme % 4 == 2:
    maison = "Serdalgia"
else:
    maison = "Serpentard"
dico[nom] = maison
ligne = f.readline()
return dico

# lecture du fichier json
def lire_json(fichier):
    with open(fichier, "r") as f:
        dico = json.load(f)
    return dico

# *** Programme principal ***
test_nb_erreurs()

# Remarque de comment par les fichiers
dico_methode = lire_txt("questionnaire_premiere_annee.txt")
dico_choixpeau = lire_json("affectation_premiere_annee.json")
print("Dictionnaire créé par ta méthode :")
print(dico_methode)

print("\nDictionnaire du Choixpeau magique :")
print(dico_choixpeau)

erreurs = nb_erreurs(dico_methode, dico_choixpeau)
pourcentage = (erreurs / len(dico_methode)) * 100

print("Nombre d'erreurs :", erreurs)
print("Pourcentage d'erreurs :", pourcentage, "%")

# annes = 4 kg

Test OK
Dictionnaire créé par ta méthode :
{'Lisa Fischer': 'Gryffondor', 'Donna Weiss': 'Serdalgia', 'Erik Montes': 'Gryffondor', 'David Washington': 'Serpentard', 'Christopher Wilson': 'Pouffouffle', 'Robert Camacho': 'Serdalgia', 'Francisco Kling': 'Pouffouffle', 'Steven Baldwin': 'Gryffondor', 'Richard Munoz': 'Serdalgia', 'Rachel Ramirez': 'Serdalgia', 'Lisa Fischer': 'Serpentard', 'Donna Weiss': 'Serpentard', 'Erik Montes': 'Serpentard', 'David Washington': 'Serpentard', 'Christopher Wilson': 'Serpentard', 'Robert Camacho': 'Serpentard', 'Francisco Kling': 'Serpentard', 'Steven Baldwin': 'Serpentard', 'Richard Munoz': 'Serpentard', 'Rachel Ramirez': 'Serpentard'}
Dictionnaire du Choixpeau magique :
{'Lisa Fischer': 'Serpentard', 'Donna Weiss': 'Serpentard', 'Erik Montes': 'Serpentard', 'David Washington': 'Serpentard', 'Christopher Wilson': 'Serpentard', 'Robert Camacho': 'Serpentard', 'Francisco Kling': 'Serpentard', 'Steven Baldwin': 'Serpentard', 'Richard Munoz': 'Serpentard', 'Rachel Ramirez': 'Serpentard'}
Nombre d'erreurs : 98
Pourcentage d'erreurs : 79.8122586451613 %
```

```
# determine si Wilson
somme = sum(reponses)
if somme % 4 == 0:
    maison = "Gryffondor"
elif somme % 4 == 1:
    maison = "Pouffouffle"
elif somme % 4 == 2:
    maison = "Serdalgia"
else:
    maison = "Serpentard"
dico[nom] = maison
ligne = f.readline()
return dico

# lecture du fichier json
def lire_json(fichier):
    with open(fichier, "r") as f:
        dico = json.load(f)
    return dico

# *** Programme principal ***
test_nb_erreurs()

# Remarque de comment par les fichiers
dico_methode = lire_txt("questionnaire_premiere_annee.txt")
dico_choixpeau = lire_json("affectation_premiere_annee.json")
print("Dictionnaire créé par ta méthode :")
print(dico_methode)

print("\nDictionnaire du Choixpeau magique :")
print(dico_choixpeau)

erreurs = nb_erreurs(dico_methode, dico_choixpeau)
pourcentage = (erreurs / len(dico_methode)) * 100

print("Nombre d'erreurs :", erreurs)
print("Pourcentage d'erreurs :", pourcentage, "%")

# annes = 4 kg

Test OK
Dictionnaire créé par ta méthode :
{'Lisa Fischer': 'Gryffondor', 'Donna Weiss': 'Serdalgia', 'Erik Montes': 'Gryffondor', 'David Washington': 'Serpentard', 'Christopher Wilson': 'Pouffouffle', 'Robert Camacho': 'Serdalgia', 'Francisco Kling': 'Pouffouffle', 'Steven Baldwin': 'Gryffondor', 'Richard Munoz': 'Serdalgia', 'Rachel Ramirez': 'Serdalgia', 'Lisa Fischer': 'Serpentard', 'Donna Weiss': 'Serpentard', 'Erik Montes': 'Serpentard', 'David Washington': 'Serpentard', 'Christopher Wilson': 'Serpentard', 'Robert Camacho': 'Serpentard', 'Francisco Kling': 'Serpentard', 'Steven Baldwin': 'Serpentard', 'Richard Munoz': 'Serpentard', 'Rachel Ramirez': 'Serpentard'}
Dictionnaire du Choixpeau magique :
{'Lisa Fischer': 'Serpentard', 'Donna Weiss': 'Serpentard', 'Erik Montes': 'Serpentard', 'David Washington': 'Serpentard', 'Christopher Wilson': 'Serpentard', 'Robert Camacho': 'Serpentard', 'Francisco Kling': 'Serpentard', 'Steven Baldwin': 'Serpentard', 'Richard Munoz': 'Serpentard', 'Rachel Ramirez': 'Serpentard'}
Nombre d'erreurs : 98
Pourcentage d'erreurs : 79.8122586451613 %
```

Question 7

Suite aux ricardements de Severus, le professeur Remus Lupin vous propose de tester la fiabilité d'une méthode de répartition aléatoire des élèves. Pour cela, répartissez aléatoirement les élèves dans les maisons et comptabiliser le nombre d'erreurs avec la répartition du choixpeau. Vous ferez la moyenne sur 100 répartitions aléatoires pour avoir une valeur plus précise. Que peut-on en conclure ?

```
import json
import random

# compte le nombre d'erreurs
def nb_erreurs(dico1, dico2):
    erreurs = 0
    élèves = list(dico1.keys())
    i = 0
    while i < len(élèves):
        if dico1[élèves[i]] != dico2[élèves[i]]:
            erreurs += 1
        i += 1
    return erreurs

# lecture du fichier json du Choixpeau
def lire_json(fichier):
    with open(fichier, "r") as f:
        dico = json.load(f)
    return dico

# fonction pour répartir aléatoirement les élèves
def repartition_alatoire(élèves):
    maisons = ["Gryffondor", "Pouffouffle", "Serdalgia", "Serpentard"]
    dico = {}
    i = 0
    while i < len(élèves):
        dico[élèves[i]] = random.choice(maisons)
        i += 1
    return dico

# *** Programme principal ***
dico_choixpeau = lire_json("affectation_premiere_annee.json")
élèves = list(dico_choixpeau.keys())

# Répéter 100 fois
i = 0
total_erreurs = 0
while i < 100:
    dico_alatoire = repartition_alatoire(élèves)
    total_erreurs += nb_erreurs(dico_alatoire, dico_choixpeau)
    i += 1

moyenne_erreurs = total_erreurs / 100
pourcentage_moyen = (moyenne_erreurs / len(élèves)) * 100

print("Moyenne d'erreurs sur 100 répartitions :", moyenne_erreurs)
print("Pourcentage moyen d'erreurs :", pourcentage_moyen, "%")

# Une méthode aléatoire place les élèves correctement environ 21 % du temps, donc 3 élèves sur 4 sont mal répartis or
# il y a 98 erreurs sur 100 d'erreurs autour de 79.8 qui revient à dire que la méthode est fiable et le professeur Rigus a raison

# annes = 4 kg

Moyenne d'erreurs sur 100 répartitions : 92.92
Pourcentage moyen d'erreurs : 74.575864516129 %
```